MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
NATIONAL TECHNICAL UNIVERSITY
“KHARKIV POLYTECHNIC INSTITUTE”
DEPARTMENT OF SOFTWARE ENGINEERING AND MANAGEMENT INFORMATION TECHNOLOGIES

Report of laboratory work № 4

Discipline «Software Engineering»
Theme: Creating of Software Requirement Specification

Executed by:
Student of group KN-226e
Budkov O. Yu.

Checked by:
Associate professor
Melnyk K. V.

Kharkiv
2026

Theme: creating of Software Requirement Specification.

Goal: to learn the basic principles of creating a Software Requirement Specification.

Tasks:

– choose a pattern for creating the SRS;

– fill in the SRS according to the previous labs;

– make all necessary actions on github.com and show the task with the updated information;

– prepare the report of the work.

Progress of work

Task 4.1 Choosing a Pattern for the SRS

A Software Requirements Specification (SRS) describes what the software must do. The pattern from the methodical recommendation, which is based on IEEE 830, was chosen. It has three parts: the purpose, the overall description and the specific requirements.

The project is planned with Scrum, so the functional requirements are written as user stories of the Product Backlog: “As a …, I want …, so that …”. Every story keeps its number from the Scrum board and has a priority, an estimate and a status. The work on the SRS is the user story US-10 “Agree on the requirements” (#71) of Sprint 4.

Task 4.2 Filling in the SRS

The SRS was filled in according to the previous labs: the formula and its limits from Lab 1, the program and its screens from Lab 2, and the Product Backlog from Lab 3. The source is the file docs/lab4/SRS.md in the repository, and the document is also stored as SRS_Function_Table_Calculator.docx and .pdf. Its full text is below.

1. Purpose

This document describes what the Function Table Calculator must do. The customer uses it to accept the product, and the developer uses it to build and test the product. Every requirement has an identifier and a way to verify it.

The requirements of the product are the user stories of the Product Backlog on GitHub. Each functional requirement below belongs to one user story, so the document and the Scrum board use the same numbers.

1.1 Definitions

Term	Meaning
x	the argument of the function
y(x)	the value of the function of Variant 3 for one x
start, end, step	the first value of x, the last value of x and the distance between two values of x
n	the upper limit of the counters in the formula, a whole number
Result table	the list of pairs x and y for every x from start to end
User story	a requirement written as “As a …, I want …, so that …”
Story points (SP)	the relative size of a user story: 1, 2, 3, 5 or 8
MoSCoW	a priority: Must, Should or Could have
Regression scenario	a recorded input with the expected output, in the folder `tests/regression`

1.2 System overview

The Function Table Calculator is a console program in C++. It calculates this function for every x from the start value to the end value with the given step:

```
y = 1/(2x) + Π (i + x),                  i = 1..n-2,            if x < 3
y = Π (j = 1..n) Σ (i = 1..n+3) (i + 1/(4j²)),                  if x ≥ 3
```

The user enters the data from the keyboard or loads it from a file, sees the result table and can save it to a file. The formula sets two limits: n must be greater than 3, because the upper limit n-2 of the first product must be greater than 1, and x = 0 has no value, because 1/(2x) divides by zero. The second branch does not contain x, so for the same n it gives the same y for every x ≥ 3.

1.3 References

1. Melnyk K. Methodical recommendation to Software Engineering laboratory practice. NTU “KhPI”, 2026.
2. IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications.
3. Budkov O. Yu. Reports of laboratory works 1–3, folder `docs` of the repository.
4. Repository of the project: https://github.com/oleksiibudkov/SE-Lab-Var3
5. Scrum board of the project: https://github.com/users/oleksiibudkov/projects/1

2. Overall description

2.1 Product perspective

The program is a standalone application. It is not a part of a larger system.

Interface	Description
System interfaces	none: the program works alone
User interfaces	a text menu in the console, in English; the user types the number of an option and presses Enter
Hardware interfaces	keyboard and screen through the console, no direct access to hardware
Software interfaces	the C++ standard library and the file system of the operating system
Communication interfaces	none: the program does not use a network
Memory constraints	the program keeps only the current data and prints the table row by row, so the table is not stored in memory

2.2 Design constraints

Operations. The user starts the program, works in the main menu and leaves it with option 0. The end of input (Ctrl+D or Ctrl+Z) also ends the program.

Site adaptation requirements. The program needs no installation or settings. Files are read from and written to the folder where the program runs.

The code is split into the layers `core`, `io` and `ui`, and a layer uses only the layers below it. The `core` layer has no console input or output.

2.3 Product functions

– Enter the data from the keyboard, with a check of every value.

– Load the data from a text file.

– Calculate y for every x and show the result table.

– Save the result table to a file.

– Show the main menu and the help.

2.4 User characteristics

The users are students and teachers. They know the formula and can work with a console. They need no programming knowledge. The program explains every value and every error.

2.5 Constraints, assumptions and dependencies

– n is a whole number from 4 to 1000.

– The table has at most 1000 rows.

– Numbers are typed with a dot: 0.5, not 0,5.

– A C++17 compiler is needed to build the program.

– The results are double precision numbers, so very big values can exceed the range of the type.

3. Specific requirements

3.1 External interface requirements

ID	Interface	Requirement
EI-1	Console	The program reads from the standard input and writes to the standard output.
EI-2	Input file	A text file with four numbers separated by spaces or new lines, in the order `start end step n`. The default name is `input.txt`.
EI-3	Output file	The text file `results.txt` in the current folder: a line with the data, an empty line and the table.

3.2 Functional requirements

The status shows the state of the user story on the Scrum board. In the column “Verification”, a name in code font is a regression scenario run by `make test`.

US-4: Enter data from the keyboard (#46)

As a user, **I want** to type the input values and get a clear message about every wrong value, **so that** I can fix a mistake at once.

Priority: Must. Estimate: 5 SP. Status: Done.

ID	Requirement	Verification
FR-1.1	The program asks for start, end, step and n, one value at a time.	`keyboard_errors`
FR-1.2	Text that is not a number is rejected with a message and an example, and the program asks again.	`keyboard_errors`
FR-1.3	An end value smaller than the start value is rejected.	`keyboard_errors`
FR-1.4	A step that is not greater than 0 is rejected.	`keyboard_errors`
FR-1.5	A value of n that is not whole, is smaller than 4 or is greater than 1000 is rejected.	`keyboard_errors`
FR-1.6	After the last value the program shows the accepted data.	`keyboard_errors`

US-5: Load data from a file (#49)

As a user, **I want** to load the input values from a file, **so that** I don't type them every time.

Priority: Should. Estimate: 3 SP. Status: Done.

ID	Requirement	Verification
FR-2.1	The user can type a file name or press Enter to use `input.txt`.	`file_ok`
FR-2.2	The values of the file pass the same checks as the values from the keyboard.	`file_bad`
FR-2.3	A missing file, a file without four numbers and a file with extra values give a message that names the problem.	`file_missing`, `file_bad`
FR-2.4	If the file is wrong, the current data does not change.	`file_bad`

US-3: See the table of y(x) (#42)

As a user, **I want** to see y for every x from the start to the end value with a chosen step, **so that** I can study how the function changes.

Priority: Must. Estimate: 5 SP. Status: Done, except FR-3.3.

ID	Requirement	Verification
FR-3.1	The program calculates y by the formula of section 1.2. For start -2, end 4, step 1 and n = 5 the values of y are -0.25, -0.5, not defined, 24.5, 60.25, 6.54826e+07 and 6.54826e+07.	`file_ok`
FR-3.2	The table shows the first value at x = start and adds the step for every next row, up to the end value.	`fractional_step`
FR-3.3	The table has at most 1000 rows. A step that gives more rows is rejected with a message.	Known deviation, see below
FR-3.4	For x = 0 the table shows “not defined”.	`file_ok`, `fractional_step`
FR-3.5	A value that exceeds the range of the type is shown as “too large”.	`big_values`
FR-3.6	Without entered data the table is not built, and the program says how to enter the data.	`no_data`

Known deviation. The current program does not meet FR-3.3 in every case. The input `0 1000 1 1000` gives 1001 rows. When start = end, the check does not limit a very small step: `1 1 1e-15 5` gives 1 000 001 rows, and `1 1 1e-300 5` prints without an end. The fix is planned as task #92 in Sprint 5.

US-6: Save the table to a file (#52)

As a user, **I want** to save the result table to a file, **so that** I can use it later.

Priority: Should. Estimate: 2 SP. Status: Done.

ID	Requirement	Verification
FR-4.1	After the table the program asks whether to save it; only y and n are accepted.	`big_values`
FR-4.2	With the answer y the program writes `results.txt` and confirms it.	`keyboard_errors`, `fractional_step`
FR-4.3	If the file cannot be created, the program shows a message and continues to work.	Manual check with a read-only folder

US-7: Use the program without a manual (#54)

As a new user, **I want** a main menu and a help screen, **so that** I understand how to work with the program.

Priority: Should. Estimate: 3 SP. Status: Done.

ID	Requirement	Verification
FR-5.1	The main menu shows the formula, the current data and the options 1–4 and 0.	`menu_errors`
FR-5.2	A wrong or empty choice gives a message, and the menu appears again.	`menu_errors`
FR-5.3	Help explains every input value, the file format and the special results.	`menu_errors`
FR-5.4	The end of input ends the program with a message and the exit code 0.	`input_ended`

US-11: Never crash on bad input (#77)

As a user, **I want** the program to handle any wrong input or file problem, **so that** it never stops unexpectedly.

Priority: Must. Estimate: 5 SP. Status: Planned, Sprint 5.

ID	Requirement	Verification
FR-6.1	Wrong values and file errors are reported with a message, not a crash.	Unit tests, Sprint 5
FR-6.2	Errors are handled with exceptions in one place.	Code review, Sprint 5

US-13: Choose the precision of the table (#85)

As a user, **I want** to choose how many digits the table shows, **so that** the results are easier to read.

Priority: Could. Estimate: 2 SP. Status: Not planned yet.

ID	Requirement	Verification
FR-7.1	The precision from 1 to 10 digits is set in the menu.	When it is planned
FR-7.2	The precision is used both on the screen and in `results.txt`.	When it is planned

US-14: Export the table to CSV (#86)

As a user, **I want** to save the table as a CSV file, **so that** I can open it in Excel.

Priority: Could. Estimate: 3 SP. Status: Not planned yet.

ID	Requirement	Verification
FR-8.1	The file has the header `x,y` and one row for every x.	When it is planned
FR-8.2	“not defined” and “too large” are kept as text.	When it is planned

3.3 Performance requirements

ID	Requirement	Measured
PR-1	The largest valid table (`0 1000 1 1000`) is shown in at most 2 seconds.	0.88 s, MacBook Air (Mac14,2), build with `make`
PR-2	The program uses at most 10 MB of memory for this table.	1.6 MB

3.4 Logical database requirements

The program has no database. It keeps one record in memory while it runs:

Field	Type	Meaning
xStart	double	the start value of x
xEnd	double	the end value of x
dx	double	the step
n	int	the upper limit of the counters

3.5 Software system attributes

ID	Attribute	Requirement	Verification
QA-1	Reliability	The same input gives the same output.	`make test`: 9 scenarios compare the output byte by byte
QA-2	Availability	The program works offline and depends on no service.	No network code in the program
QA-3	Security	The program collects no personal data and uses no network. `results.txt` is replaced without a warning.	Code review
QA-4	Maintainability	The code is split into the layers `core`, `io` and `ui`, and it builds without warnings with `-Wall -Wextra -pedantic`.	`make` shows no warnings
QA-5	Portability	The code uses only standard C++17. It builds with `make` and with CMake.	Checked on macOS with Apple clang; the build in Visual Studio on Windows is not checked yet

3.6 Environment characteristics

Hardware. Any computer that runs a C++ compiler. The program needs less than 10 MB of memory.

Peripherals. A keyboard and a screen. A printer is not needed.

4. Traceability

User story	Requirements	Priority	SP	Sprint	Status
US-1, US-2	analysis and UML models: limits of n and x in section 1.2	Must	8	1	Done
US-3	FR-3.1–FR-3.6	Must	5	2	Done, except FR-3.3
US-4	FR-1.1–FR-1.6	Must	5	2	Done
US-5	FR-2.1–FR-2.4	Should	3	2	Done
US-6	FR-4.1–FR-4.3	Should	2	2	Done
US-7	FR-5.1–FR-5.4	Should	3	2	Done
US-9	QA-4, QA-5	Should	5	3	Done
US-11	FR-6.1, FR-6.2, FR-3.3	Must	5	5	Planned
US-12	unit tests for FR-3 and the input checks	Must	8	5	Planned
US-13	FR-7.1, FR-7.2	Could	2	not planned	Backlog
US-14	FR-8.1, FR-8.2	Could	3	not planned	Backlog

Task 4.3 Actions on github.com

Sprint 3 was closed with a velocity of 10 story points, and US-10 was taken into work. It was started on 5 November, before the planned start of Sprint 4 on 9 November. Two tasks were added during the work: #91 for the GitHub actions of this lab and #92 for the defect found in the row limit. The state of the sprint is shown on the Scrum board (Figure 4.1).

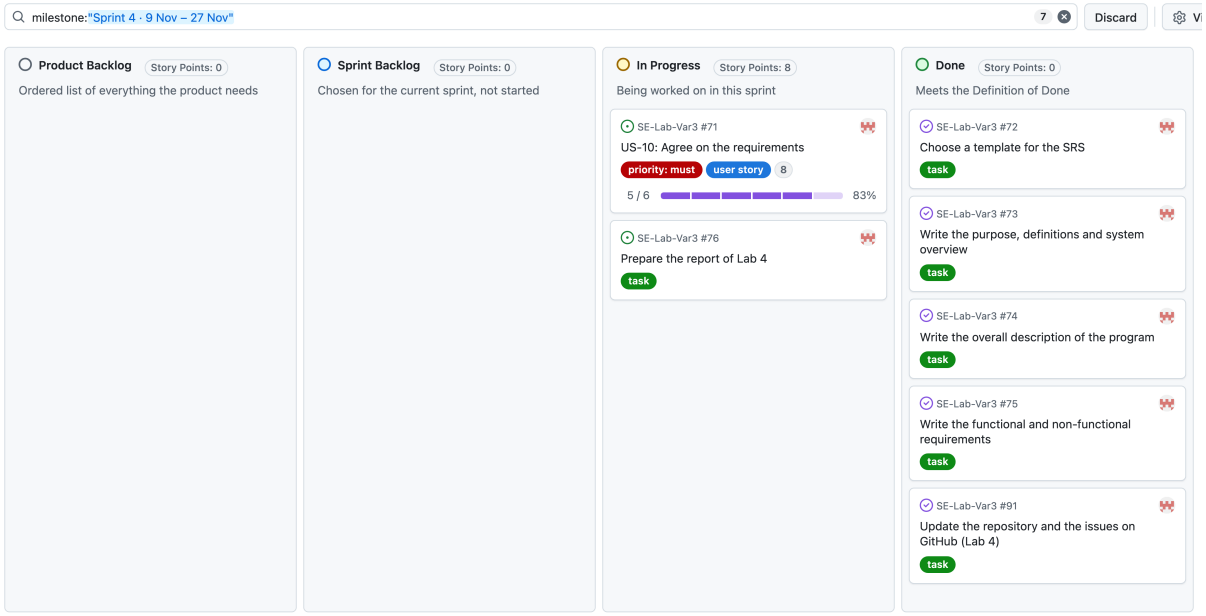


Figure 4.1 – Scrum board filtered by Sprint 4

The page of US-10 shows the met acceptance criteria and the tasks of the story (Figure 4.2). The last open task is the report of this lab (#76). The task of writing the requirements is completed (Figure 4.3): it was closed by its commit when the pull request was merged, and the task page keeps the log of the work.

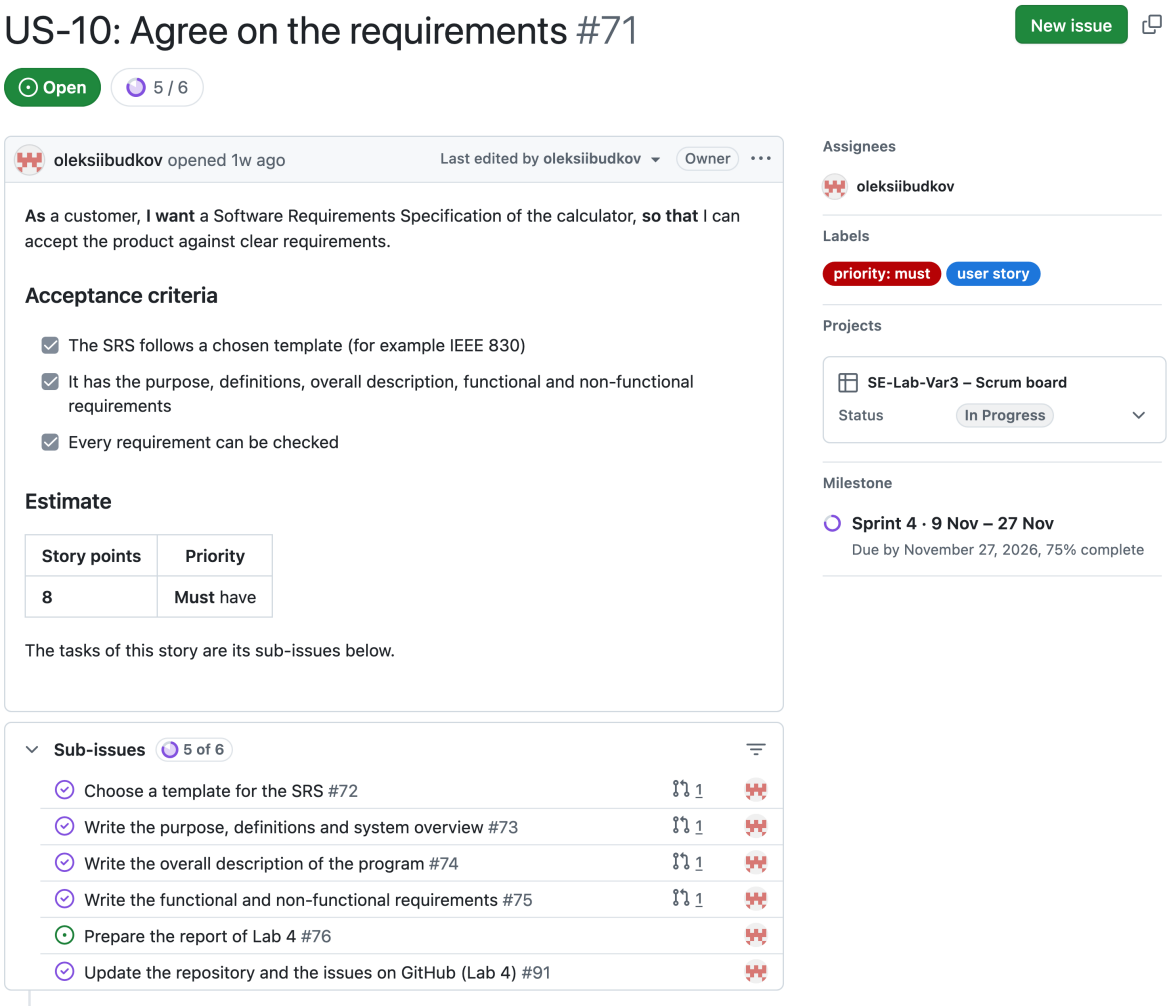


Figure 4.2 – User story US-10 with its acceptance criteria and tasks

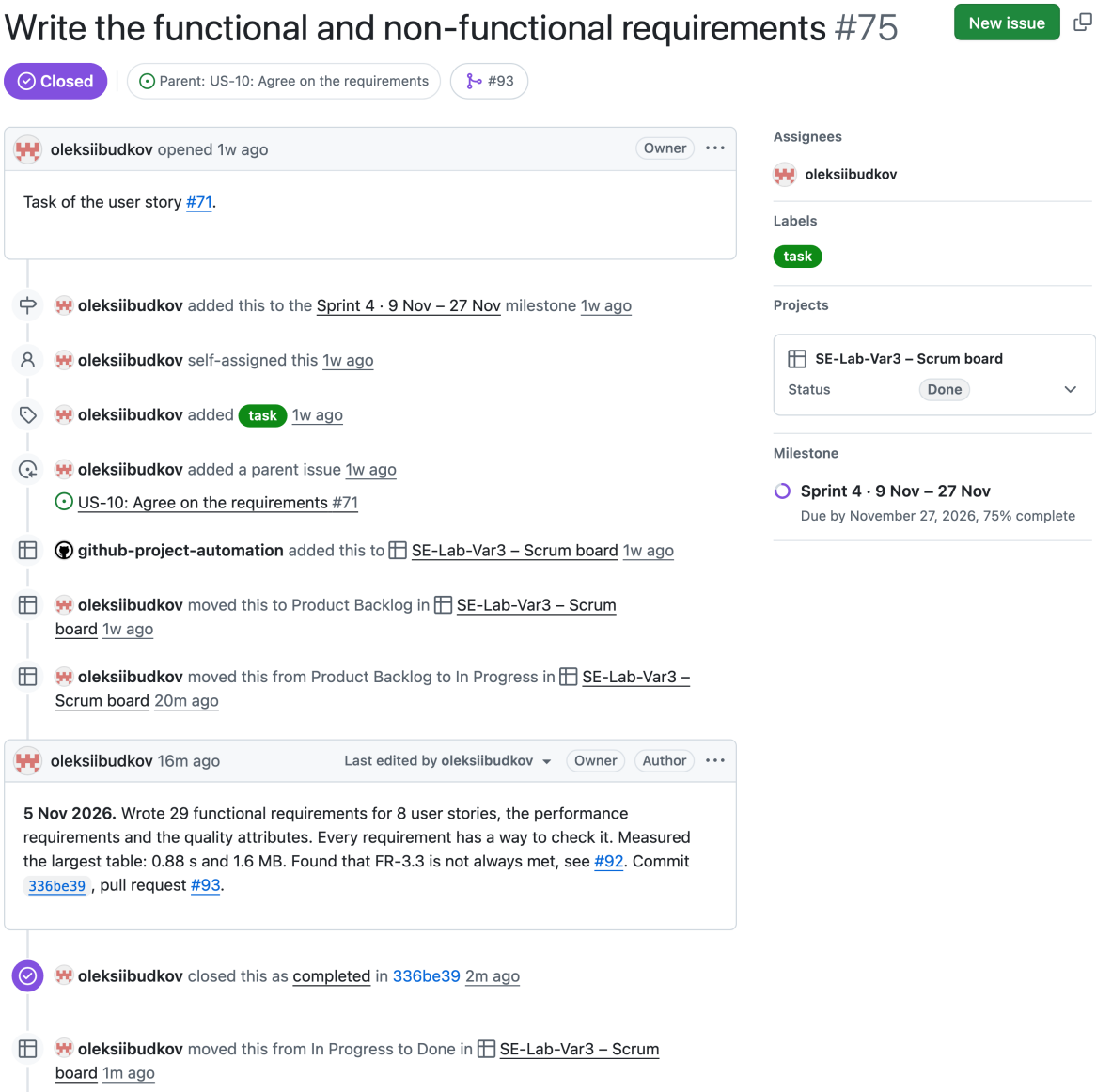


Figure 4.3 – Completed task “Write the functional and non-functional requirements”

The SRS was written in the branch feature/us-10-srs, with one commit for every task, and merged into the main branch through the pull request #93 (Figures 4.4–4.5). The work is logged in Table 4.1 and in the comments of the tasks.

US-10: Add the Software Requirements Specification #93

Merged oleksiibudkov merged 6 commits into `main` from `feature/us-10-srs` 2 minutes ago

Conversation 0 · Commits 6 · Checks 0 · Files changed 5 · +261

oleksiibudkov commented 17 minutes ago — Owner

What

The Software Requirements Specification of the calculator for user story US-10.

- `docs/lab4/SRS.md` is the source; the `.docx` and `.pdf` are generated from it.
- The structure follows the pattern from Lab 4: purpose, overall description, specific requirements.
- Functional requirements are grouped by the user stories of the Product Backlog. Every requirement has a way to check it, and section 4 links stories, requirements and sprints.
- `tools/github_setup.sh` now includes tasks Update the repository and the issues on GitHub (Lab 4) #91 and Fix the row limit check of the result table #92.

Found while writing

The row limit FR-3.3 is not always met (for example, `1 1 1e-300 5` prints without an end). The SRS records this as a known deviation, and the fix is planned in #92. This pull request does not change the code.

How it was checked

- `make` shows no warnings, and `make test` gives 9 passed. CMake build: no warnings, 9 passed.
- Every regression scenario named in the SRS was checked to show the described behaviour; PR-1 and PR-2 were measured.

Sprint backlog

Closes #72, closes #73, closes #74, closes #75

Reviewers: No reviews

Assignees: oleksiibudkov

Labels: user story

Projects: None yet

Milestone: Sprint 4 · 9 Nov – 27 Nov

Development: Successfully merging this pull request may close these issues.
- Write the functional and non-functional requirem...
- Write the overall description of the program
- Write the purpose, definitions and system overvi...
- Choose a template for the SRS

1 participant

Figure 4.4 – Pull request of the SRS

Commits

main · All users · All time

Commits on Nov 5, 2026

Merge pull request #93 from oleksiibudkov/feature/us-10-srs oleksiibudkov authored 2 minutes ago	Verified	18f7097
Add the SRS as a document and link it from README oleksiibudkov committed 17 minutes ago		b9eeae5
Add the new Lab 4 and bug tasks to github_setup.sh oleksiibudkov committed 17 minutes ago		f7071a5
Write the functional and non-functional requirements of the SRS oleksiibudkov committed 18 minutes ago		336be39
Write the overall description of the program in the SRS oleksiibudkov committed 19 minutes ago		e9a88c6
Write the purpose, definitions and system overview of the SRS oleksiibudkov committed 19 minutes ago		9b147c9
Add the SRS template based on the pattern of the methodical recommendation oleksiibudkov committed 19 minutes ago		23ad4b3

Commits on Oct 30, 2026

Merge pull request #90 from oleksiibudkov/Super-branch oleksiibudkov authored last week	Verified	18ec569
This super comment oleksiibudkov committed last week		0be5a43

Figure 4.5 – History of commits

Table 4.1 – Work log of US-10, 5 November 2026

Issue	Action	Result
Sprint 3	Sprint Review, Sprint Planning	Sprint closed: 14 of 14 issues, 10 SP; US-10 moved to In Progress
#72	Chose the pattern and added the template	Commit 23ad4b3
#73	Wrote the purpose, definitions and system overview	Commit 9b147c9
#74	Wrote the overall description	Commit e9a88c6
#75	Wrote the functional and non-functional requirements	Commit 336be39; 29 functional requirements
#92	Recorded the defect of the row limit	Task in Sprint 5, under US-11
#91	Updated the script and the board, merged pull request #93	Tasks #72–#75 moved to Done
#76	Prepared this report	In Progress until the report is merged

Conclusions

During this laboratory work we got skills and knowledge about creating a Software Requirements Specification. We chose a pattern based on IEEE 830 and filled it in according to the previous labs. We wrote the functional requirements as user stories with a priority, an estimate and a way to check every requirement, and linked them to the Scrum board. Measuring the program for the SRS showed that the limit of 1000 rows is not always met, so the defect was added to the backlog for Sprint 5. We made the work on GitHub with a feature branch and a pull request, and logged it in the tasks and on the board.